

Implementasi Awal CRUD: Campus Event Manager

Topik: Campus Event Management System

Kelompok: 11

Github: github.com/Dzaky-Ar/DBProject_Group11_CampusEventManager

1. Pendahuluan

Dari percobaan yang telah dibuat, akan diperlihatkan salah satu workflow dari penerapan operasi CRUD (Create, Read, Update, Delete), yaitu fitur login dan registrasi. Analisis dilakukan berdasarkan implementasi kode pada berkas Login_Page.php, Register_Page.php, dan Mapping.php yang berperan dalam proses autentikasi pengguna. Percobaan ini juga akan memanfaatkan SQL Injection atau biasa disebut SQLi untuk penggunaan kedepannya.

```
root/
├── Configuration/           # Berisi konfigurasi server dan database
│   └── config.php          # File koneksi database
├── Main_menu/              # Modul autentikasi dan menu utama
│   ├── Login_Page.php      # Halaman login pengguna
│   ├── Register_Page.php   # Halaman registrasi pengguna
│   └── Mapping.php         # Logika autentikasi dan proses CRUD
└── Dashbaord/              # Halaman dashboard setelah login
```

2. Operasi CRUD pada Form Login dan Register

- Create (Registrasi Pengguna)
 - Deskripsi

Operasi Create terjadi ketika pengguna melakukan pendaftaran akun melalui form registrasi. Pada tahap ini, sistem menyimpan data pengguna baru ke dalam basis data, khususnya ke tabel User, dan dalam kondisi tertentu juga ke tabel Organizer atau Verificator.

The image shows a registration form titled "Register" in orange text. The form is set against a white background with a teal border. It contains the following fields and elements:

- Username:** A text input field containing the value "nanda".
- Email:** A text input field containing the value "nanda@gmail.com".
- Password:** A text input field containing six asterisks "*****".
- Status:** A dropdown menu with "Guest/User" selected and a downward arrow icon.
- Confirm:** An orange button with the text "Confirm".
- Footer:** A link that says "Already have an account? [Login](#)".

- Implementasi

Proses registrasi dilakukan melalui beberapa tahapan berikut:

1. Pengguna mengisi form registrasi pada Register_Page.php.
2. Data dikirim ke Mapping.php menggunakan metode HTTP POST.
3. Method register() dalam class Authentication menangani proses registrasi, yang meliputi:
 - a. Validasi input dasar (email, password, dan status pengguna).
 - b. Pengecekan apakah email sudah terdaftar melalui query SELECT.
 - c. Hashing password menggunakan fungsi password_hash() untuk menjaga keamanan kredensial.
 - d. Penyimpanan data pengguna ke tabel User menggunakan prepared statement.
 - e. Jika pengguna berstatus organizer, data tambahan disimpan ke tabel Organizer.

- Read (Login Pengguna)

- Deskripsi

Operasi Read terjadi pada proses login, di mana sistem membaca data pengguna dari basis data untuk memverifikasi kredensial yang dimasukkan.

- Implementasi

Tahapan proses login adalah sebagai berikut:

1. Pengguna mengisi email dan password pada Login_Page.php.
2. Data dikirim ke Mapping.php.
3. Method login() dalam class Authentication:
 - a. Menjalankan query SELECT untuk mengambil data pengguna berdasarkan email.
 - b. Memverifikasi password menggunakan fungsi password_verify().
 - c. Jika autentikasi berhasil, sistem membuat session dan mengarahkan pengguna ke halaman sesuai dengan statusnya.

Login

Email successfully registered. Please login.

Email
nanda@gmail.com

Password
.....

Login

Don't have an account? [Register](#)

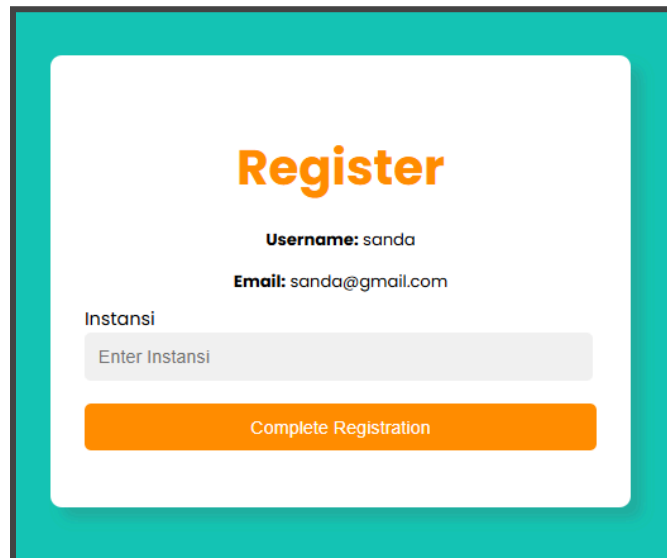
- Update untuk Organizer

- Deskripsi

Operasi Update tidak secara eksplisit muncul dalam form login dan register. Namun, proses registrasi organizer dapat dikategorikan sebagai operasi update secara konseptual, karena melibatkan perubahan status atau penambahan informasi pengguna setelah registrasi awal.

- Implementasi

Proses ini terjadi saat sistem menambahkan data lanjutan ke tabel Organizer berdasarkan data yang tersimpan dalam session pengguna. Meskipun secara teknis berupa insert data baru, proses ini merepresentasikan pembaruan status pengguna dalam sistem.



The image shows a registration form titled "Register" in orange text. Below the title, it displays the filled-in fields: "Username: sanda" and "Email: sanda@gmail.com". There is a label "Instansi" above a text input field containing the placeholder "Enter Instansi". At the bottom of the form is an orange button labeled "Complete Registration". The entire form is enclosed in a teal border.

- Delete

- Deskripsi

Operasi Delete tidak diterapkan secara langsung pada proses login dan registrasi pengguna. Akan tetapi, operasi ini diimplementasikan pada user page, khususnya pada fitur pembatalan registrasi event (cancel submission). Fitur ini memungkinkan pengguna yang telah terdaftar pada suatu event untuk membatalkan partisipasinya, sehingga data registrasi terkait dihapus dari basis data.

Judul	Event Name	File	Update	Action
sample	Malam Apresiasi	View PDF	Update	Cancel

Dari tabel, dengan menggunakan fitur action, spesifiknya tombol cancel ini akan menghapus submisi atau presensi dari user yang telah mendaftarkan diri dalam suatu event.

3. Implementasi Kode Program

Login_Page

```
<?php
session_start();

// Get possible flash messages (either from GET or session flash)
$successGet = $_GET['success'] ?? "";
$flash = $_SESSION['flash_message'] ?? "";
$loginError = $_SESSION['login_error'] ?? "";

if (!empty($flash)) {
    // prefer session flash if exists
    $message = $flash;
    unset($_SESSION['flash_message']);
} else {
    if ($successGet === 'registered') {
        $message = "Email successfully registered. Please login.";
    } elseif ($successGet === 'organizer') {
        $message = "Registration is completed. Please login.";
    } else {
        $message = "";
    }
}

// Clear only login_error after reading
if (!empty($loginError)) {
    unset($_SESSION['login_error']);
}
?>
<!DOCTYPE html>
```

```

<html>
<head>
  <title>Login</title>
  <link rel="stylesheet" href="../../styling/style_loginRegister.css">
  <link rel="icon" href="../../images/logo web.png">
</head>
<body>
  <div class="container">
    <div class="form-box active">
      <form action="Mapping.php" method="POST">
        <h2 id="header">Login</h2>

        <?php if (!empty($message)): ?>
          <p class="success-message"><?= htmlspecialchars($message); ?></p>
        <?php endif; ?>

        <?php if (!empty($loginError)): ?>
          <p class="error-message"><?= htmlspecialchars($loginError); ?></p>
        <?php endif; ?>

        <label>Email</label>
        <input type="email" name="Email" placeholder="Enter email" required>

        <label>Password</label>
        <input type="password" name="Password" placeholder="Enter Password"
required>

        <button type="submit" name="login">Login</button>
        <p>Don't have an account? <a href="Register_Page.php">Register</a></p>
      </form>
    </div>
  </div>
</body>
</html>

```

Register_Page

```
<?php
session_start();

$registerError = $_SESSION['register_error'] ?? "";
$registerSuccess = $_SESSION['register_success'] ?? "";

// Clear after reading so it behaves like a flash
unset($_SESSION['register_error'], $_SESSION['register_success']);
?>
<!DOCTYPE html>
<html>
<head>
    <title>Register</title>
    <link rel="stylesheet" href="../../styling/style_loginRegister.css">
    <link rel="icon" href="../../images/logo web.png">
</head>
<body>
    <div class="container">
        <div class="form-box active">
            <form action="Mapping.php" method="POST">
                <h2 id="header">Register</h2>

                <?php if (!empty($registerError)): ?>
                    <p class="error-message"><?= htmlspecialchars($registerError); ?></p>
                <?php endif; ?>

                <?php if (!empty($registerSuccess)): ?>
                    <p class="success-message"><?= htmlspecialchars($registerSuccess);
?></p>
                <?php endif; ?>

                <label>Username</label>
                <input type="text" name="User_name" placeholder="Enter Username"
required>

                <label>Email</label>
                <input type="email" name="Email" placeholder="Enter Email" required>

                <label>Password</label>
                <input type="password" name="Password" placeholder="Enter Password"
required>

                <label>Status</label>
                <select name="Status_user" id="Status_user" required>
```

```

        <option value="">--Select Status--</option>
        <option value="1">Verificator </option>
        <option value="2">Organizer</option>
        <option value="3">Guest/User</option>
    </select>

    <button type="submit" name="register">Confirm</button>
    <p>Already have an account? <a href="Login_Page.php">Login</a></p>
</form>
</div>
</div>
</body>
</html>

```

Mapping.php

```

<?php
session_start();
require_once "../Configuration/config.php"; // must set $conn (mysqli)

// Simple helper - validate required POST keys
function require_keys($arr, $keys) {
    foreach ($keys as $k) {
        if (!isset($arr[$k]) || $arr[$k] === "") return false;
    }
    return true;
}

class Authentication {
    private $conn;
    public function __construct($conn) {
        $this->conn = $conn;
    }

    // register
    public function register($name, $email, $password_raw, $status) {
        // basic server-side validation
        if (empty($name) || empty($email) || empty($password_raw) || empty($status)) {
            $_SESSION['register_error'] = "All fields are required.";
            header("Location: Register_Page.php");
            exit();
        }
    }
}

```



```

    }

    // check duplicate email
    $stmt = $this->conn->prepare("SELECT Email FROM User WHERE Email =
?");
    $stmt->bind_param("s", $email);
    $stmt->execute();
    $res = $stmt->get_result();
    $stmt->close();

    if ($res && $res->num_rows > 0) {
        $_SESSION['register_error'] = "Email is already registered!";
        header("Location: Register_Page.php");
        exit();
    }

    $password_hashed = password_hash($password_raw, PASSWORD_DEFAULT);

    if ($status === '2') {
        // Save all registration data temporarily in session associative array
        $_SESSION['reg_data'] = [
            'User_name' => $name,
            'Email' => $email,
            'Password' => $password_hashed,
            'Status_user' => $status
        ];
        // Redirect to organizer details page to fill instansi
        header("Location: Organizer_Details.php");
        exit();
    }

    // For other statuses, insert immediately
    if ($this->insertUser($name, $email, $password_hashed, $status)) {
        // If verifactor status, add to verifactor table
        if ($status === '1') {
            $this->insertVerifactor($name, $email);
        }

        // redirect to login with success
        header("Location: Login_Page.php?success=registered");
        exit();
    } else {
        $_SESSION['register_error'] = "Registration failed. Try again.";
        header("Location: Register_Page.php");
        exit();
    }
}

```

```

    }

    // insert user helper
    private function insertUser($name, $email, $password, $status) {
        $stmt = $this->conn->prepare(
            "INSERT INTO User (User_name, Email, Password, Status_user) VALUES (?,
?, ?, ?)"
        );
        if (!$stmt) return false;
        $stmt->bind_param("ssss", $name, $email, $password, $status);
        $ok = $stmt->execute();
        $stmt->close();
        return $ok;
    }

    private function insertVerificator($name, $email) {
        $stmt = $this->conn->prepare(
            "INSERT INTO Verificator (Nama_PJ, Email) VALUES (?, ?)"
        );
        if (!$stmt) return false;
        $stmt->bind_param("ss", $name, $email);
        $ok = $stmt->execute();
        $stmt->close();
        return $ok;
    }

    // organizer details submission
    public function addOrganizerDetails($instansi) {
        // ensure reg_data exists
        if (!isset($_SESSION['reg_data']) || !is_array($_SESSION['reg_data'])) {
            $_SESSION['organizer_error'] = "No pending organizer registration found.";
            header("Location: Register_Page.php");
            exit();
        }

        $reg = $_SESSION['reg_data'];
        $name = $reg['User_name'];
        $email = $reg['Email'];
        $password = $reg['Password'];
        $status = $reg['Status_user']; // should be '4'

        // insert user
        $okUser = $this->insertUser($name, $email, $password, $status);

        if (!$okUser) {
            $_SESSION['organizer_error'] = "Failed to create user account. Try again.";

```

```

        header("Location: Organizer_Details.php");
        exit();
    }

    // insert organizer
    $stmt = $this->conn->prepare(
        "INSERT INTO Organizer (Nama_organizer, Email, Instansi) VALUES (?, ?,
    ?)"
    );
    if (!$stmt) {
        $_SESSION['organizer_error'] = "Database error (organizer insert).";
        header("Location: Organizer_Details.php");
        exit();
    }
    $stmt->bind_param("sss", $name, $email, $instansi);
    $okOrg = $stmt->execute();
    $stmt->close();

    if (!$okOrg) {
        // optionally rollback user insertion if your DB supports transactions; for now
    set error
        $_SESSION['organizer_error'] = "Failed to save organizer details.";
        header("Location: Organizer_Details.php");
        exit();
    }

    // success: clear temporary reg_data and redirect to login with success
    unset($_SESSION['reg_data']);
    header("Location: Login_Page.php?success=organizer");
    exit();
}

// login
public function login($email, $password_raw) {
    if (empty($email) || empty($password_raw)) {
        $_SESSION['login_error'] = "Email & password required.";
        header("Location: Login_Page.php");
        exit();
    }

    $stmt = $this->conn->prepare("SELECT * FROM User WHERE Email = ?");
    $stmt->bind_param("s", $email);
    $stmt->execute();
    $res = $stmt->get_result();
    $stmt->close();

```

```

        if ($res && $res->num_rows > 0) {
            $user = $res->fetch_assoc();

            // If organizer, ensure organizer details exist
            if ($user['Status_user'] == '2') {
                $stmt2 = $this->conn->prepare("SELECT * FROM Organizer WHERE
Email = ?");
                $stmt2->bind_param("s", $email);
                $stmt2->execute();
                $orgRes = $stmt2->get_result();
                $stmt2->close();

                if (!$orgRes || $orgRes->num_rows == 0) {
                    $_SESSION['login_error'] = "Please complete your organizer details
first.";
                    header("Location: Login_Page.php");
                    exit();
                }
            }

            if (password_verify($password_raw, $user['Password'])) {
                // set session user data
                $_SESSION['User_name'] = $user['User_name'];
                $_SESSION['Email'] = $user['Email'];
                $_SESSION['Status_user'] = $user['Status_user'];

                // redirect based on status
                switch ($user['Status_user']) {
                    case '1':
                        header("Location:
../Dashboard/Verificator/Verificator_Dashboard.php");
                        exit();
                    case '2':
                        header("Location: ../Dashboard/Organizer/Organizer_Dashboard.php");
                        exit();
                    case '3':
                        header("Location: ../Dashboard/User/User_Dashboard.php");
                        exit();
                }
            }
        }

        $_SESSION['login_error'] = "Invalid email or password.";
        header("Location: Login_Page.php");
        exit();
    }
}

```

```

}

// Instantiate
$auth = new Authentication($conn);

// POST handlers
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    if (isset($_POST['register'])) {
        // sanitize minimal
        $name = trim($_POST['User_name'] ?? '');
        $email = trim($_POST['Email'] ?? '');
        $password = $_POST['Password'] ?? '';
        $status = $_POST['Status_user'] ?? '';

        $auth->register($name, $email, $password, $status);
    }

    if (isset($_POST['organizer_details'])) {
        $instansi = trim($_POST['instansi'] ?? '');
        if ($instansi === "") {
            $_SESSION['organizer_error'] = "Instansi is required.";
            header("Location: Organizer_Details.php");
            exit();
        }
        $auth->addOrganizerDetails($instansi);
    }

    if (isset($_POST['login'])) {
        $email = trim($_POST['Email'] ?? '');
        $password = $_POST['Password'] ?? '';
        $auth->login($email, $password);
    }
}
}

```

Organizer_Details

```
<?php
session_start();

// Ensure we have pending registration data
if (!isset($_SESSION['reg_data']) || !is_array($_SESSION['reg_data'])) {
    header("Location: Register_Page.php");
    exit();
}

$reg = $_SESSION['reg_data'];

$organizerError = $_SESSION['organizer_error'] ?? "";
$organizerSuccess = $_SESSION['organizer_success'] ?? "";

// Clear flash after reading
unset($_SESSION['organizer_error'], $_SESSION['organizer_success']);
?>
<!DOCTYPE html>
<html>
<head>
    <title>Register</title>
    <link rel="stylesheet" href="../../styling/style_loginRegister.css">
    <link rel="icon" href="../../images/logo web.png">
</head>
<body>
    <div class="container">
        <div class="form-box active">
            <form action="Mapping.php" method="POST">
                <h2 id="header">Register</h2>

                <?php if (!empty($organizerError)): ?>
                    <p class="error-message"><?= htmlspecialchars($organizerError); ?></p>
                <?php endif; ?>

                <?php if (!empty($organizerSuccess)): ?>
                    <p class="success-message"><?= htmlspecialchars($organizerSuccess);
?></p>
                <?php endif; ?>

                <p><strong>Username:</strong> <?= htmlspecialchars($reg['User_name']);
?></p>
                <p><strong>Email:</strong> <?= htmlspecialchars($reg['Email']); ?></p>
```

```

        <label>Instansi</label>
        <input type="text" name="instansi" placeholder="Enter Instansi" required>

        <button type="submit" name="organizer_details">Complete
Registration</button>
    </form>
</div>
</div>
</body>
</html>

```

Cancel_Submission

```

<?php
session_start();
require_once "../Configuration/config.php";

if (isset($_POST["cancel"])) {
    $registration_id = $_POST["Registration_id"];
    $email = $_SESSION['Email'] ?? "";

    if ($email) {
        $sql = "DELETE FROM Registration WHERE Registration_ID = ? AND Email = ?";
        $stmt = $conn->prepare($sql);
        $stmt->bind_param("is", $registration_id, $email);
        if ($stmt->execute()) {
            $_SESSION['success_message'] = "Submission cancelled successfully!";
        } else {
            $_SESSION['error_message'] = "Error cancelling submission.";
        }
        $stmt->close();
    } else {
        $_SESSION['error_message'] = "User not logged in.";
    }

    header("Location: RecordSubmissions.php");
    exit();
}
?>

```

